

# Neuro-Evolved Heuristics for Variable Gapped Common Subsequence Identification

Marko Djukanović<sup>1, 5</sup>   Christian Blum<sup>2</sup>   Aleksandar Kartelj<sup>3</sup>   Sašo  
Džeroski<sup>4</sup>   Žiga Zebec<sup>5</sup>

<sup>1</sup>University of Nova Gorica, Nova Gorica, Slovenia

<sup>2</sup>Artificial Intelligence Research Institute (IIIA-CSIC), Barcelona, Spain

<sup>3</sup>Faculty of Mathematics, University of Belgrade, Belgrade, Serbia

<sup>4</sup>Jožef Stefan Institute, Ljubljana, Slovenia

<sup>5</sup>Institute of Information Sciences (IZUM), Maribor, Slovenia

- OPTIMA 2026: XVII International Conference Optimization and Applications,  
Petrovac, Montenegro –  
– September 2026 –



Sofinancira  
Evropska unija



**SMASH**  
machine learning for science and humanities  
postdoctoral program

# Introduction

- Objects we deal with: sequences (strings) over finite alphabet  $\Sigma$ 
  - DNA/RNA sequences over  $\{A, T, G, C/U\}$
  - Proteins over 20 (canonical) amino acids:  $\{A, C, D, E, P, Q, \dots\}$
- One of central tasks in computational biology:
  - sequence comparison, finding **common motifs** between sequences
  - how structural similarity affects sequence functionality
- Motifs can be **subsequences**: **Longest common subsequence problem** variants (studied 50 years already)

# Longest common subsequence problem (LCSP)

## Definition (LCSP)

**Input:** Given an arbitrary set of sequences  $S = \{s_1, \dots, s_m\}$

**Task:** Find a subsequence  $s$  **common** for all sequences from  $S$  with **maximum** possible length ( $|s|$ ).

## LCS: Literature & Problem Variants

- For  $m = 2$ , polynomially solvable, e.g. Dynamic programming
- When  $m$  arbitrary large, it is **NP-hard**: approximation, meta-heuristic, exact approaches

**Problem-related practical variants:** Repetition-free, Filled LCS problem, **Gapped LCS problem**

# The gapped LCS problem (Peng and Yang, 2012)

## Definition (Gapped sequence)

Given is a sequence  $s$  and a function  $G_s: \{1, \dots, |s|\} \mapsto \mathbb{N}$ . A pair  $(s, G_s)$  is called a gapped sequence.

( $G_s$ : To each position of seq.  $s$ , one number assigned)

## Definition (Gapped subsequence)

Sequence  $\tilde{s}$  is a **gapped subsequence** of  $(s, G_s)$  iff

- $\tilde{s}$  is a **subsequence** of  $s$
- the gap constraint  $G_s$  fulfilled resp. the *positions of embedding*  $\tilde{s}$  in  $s$ :
  - consecutive embedding positions respect gap distances (no more than the **assigned number of letters** inbetween respected)

# Problem definition

## Example

$s = \text{AATTGC}$ ,  $G_s(\cdot) = 1$ ,  $\tilde{s} = \text{ATG}$

- the embedding **AATTGC** (**valid** gapped subsequence)
- the embedding **AATTGC** (**invalid** gapped subsequence)

Definition (The multiple (variable) gapped LCS problem – MVGLCSP)

**Input:** Given is a set of  $m$  gapped sequences  $\{(s_i, G_{s_i})\}_{i=1}^m$ .

**Task:** Find the longest sequence  $\tilde{s}$  so that

- $\tilde{s}$  is **common subsequence** of each  $s_i$
- $\tilde{s}$  **fulfills all gap constraints**  $G_{s_i}$  ( $i = 1, \dots, m$ )

# Literature & Motivation for VGLCSP

- Peng and Yang (2012, 2014):  $m = 2$  version by **three dynamic programming** (DP) approaches
- Manea et al. (2024): complexity analysis investigated
  - **NP-hard** under arbitrary large  $m$
- Djukanovic et al. (2026): the first approach for  $m$  arbitrary: **Iterative Beam search approach**

## Motivation:

- **Genetics and molecular biology**: DNA/protein comparison analysis—distances between residues must be respected
- **Time-series analysis**: in settings where events are required to occur within specified temporal delays (Lainscsek et al. (2015))

# Methodology by Djukanovic et al. (2026)

- Based on the **state space graph formulation** of the problem
- **Nodes** corresp. to partial solutions and resp. subproblems to be solved
  - transitions: valid extension of partial solutions (by one letter)
- **Gap constraints**: incorporated to cut-off invalid extensions (edges) among the LCS extensions immediately
- **Many root/source nodes** in the state space—the position of leading letter in each common gapped subsequence free to select
  - root state space subgraph (formulation) w.r.t. starting source (match)

# Methodology: main components roughly

- Generation of longest solution for a root (sub-)space is generally hard  
⇒ need an approx. approach for evaluation (**Beam search – limited BFS**)
- The root-state-space formulation may involve disconnected components generation ⇒ needs **iterative beam search** to explore all space

⇒ **Iterative Multi Source Beam Search (IMSBS) approach.**

# Heuristic guidances in IMSBS

The best literature heuristic to guide BS and prioritize root node:

- ①  $UB_2$ : *Character Frequency Alignment* score

Experimental evaluation:

- IMSBS able to efficiently run on large synthetic instances
- **Weaknesses:** the performance of  $UB_2$  seems to significantly drop for large  $n$  ( $n = 500$ ) and small  $|\Sigma|$  ( $|\Sigma| = 2$ )

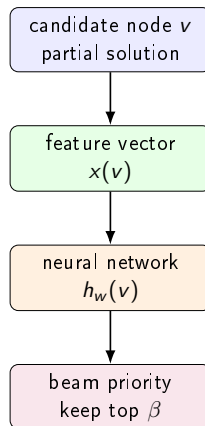
⇒ **Construction of a new heuristic guidance**

# Designing advanced heuristic guidances: main idea

- IMSBS needs a better score to decide which candid nodes stay in the beam
- We learn a **new node evaluator**: a function assigning a promise score to a partial solution

## Role of the learned heuristic

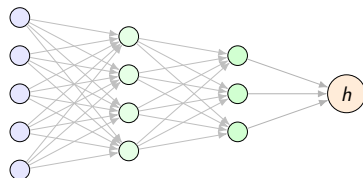
It does **not** construct the solution directly; it ranks candidate states during search.



# Neural network basics: MLP as a scoring function

- We use a **feed-forward multilayer perceptron** (MLP).
- Input: **compact numerical description** of the state  $v$ .
- Hidden layers transform features into nonlinear combinations.
- Output: a **single heuristic score**  $h_w(v)$ .

features    hidden + activation funcs    score



# State features for VGLCS node evaluation

For a state  $\mathbf{v} = (p^v, l^v)$  ( $p^v$ : vector of suffix positions of subproblem to be solved,  $l^v$ : length of partial solution), build a size-independent features:

## Search progress

- normalize positions:  
 $F_i^v = p_i^v / |s_i|$
- max, min, mean, std of  $F^v$
- current length  $l^v$

## Gap flexibility

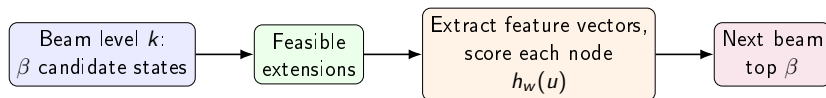
- $G_i^v = G_{s_i}[p_i^v, \dots]$
- max, min, mean, std
- describes how restrictive future extensions are

## Global context

- alphabet size  $|\Sigma|$
- number of sequences  $m$
- standardized before training

**Feature sets tested:**  $F_1 = \text{progress} + \text{gaps} + \text{length}$  (9);  $F_2 = F_1 + |\Sigma|$  (10);  
 $F_3 = F_2 + m$  (11).

# How the learned evaluator is used inside IMSBS



## Important note

Learning is **offline**; during the final run the NN is only a fast scoring function.

# Why not standard supervised backpropagation?

## Obstacle

- Exact optimal trajectories unavailable for the general case
- Search quality non-differentiable: one ranking decision affect many later states

## Solution: neuro-evolution

- Treat all NN weights as **one vector**  $w$ .
- Evaluate  $w$  by running (inexpensive) IMSBS on training (32) instances.
- **Fitness** = average subsequence length found

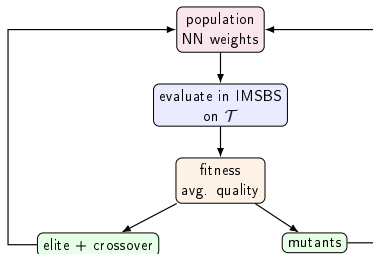
$$\max_w \frac{1}{|\mathcal{T}|} \sum_{I \in \mathcal{T}} f(\text{IMSBS}(I, h_w))$$

# Learning weights with Biased Random Key GA

- Individual = complete NN weight vector  $w$ .
- Population of weights, evolution
- ...

## Parameters used

$n_{pop} = 20$ ,  $n_{elite} = 1$ ,  $n_{mutants} = 7$ ,  
 $p_{bias} = 0.5$ .



# Rank-based ensemble: learned + analytical guidance

- Preliminary tests: the learned heuristic strong, but not uniformly best
- Hand-crafted upper bounds useful, especially in some small- $m$  cases
- Raw scores not directly comparable, **rankings** rather than scores combined

## Combined rank

$$r(v) = \alpha r_{h_{w^*}}(v) + (1 - \alpha) r_{UB_2}(v).$$

Lower rank  $r(v)$  means higher priority in the beam.

$\Rightarrow$   $UB_2$  replaced by  $r(\cdot)$  : **Ensemble-IMSBS approach**.

# What was tuned? Experimental design

## IMSBS parameters

- tuned by irace (parameters from the literature:  $\beta = 5000$ )

## Training protocol

- IMSBS utilized  $\beta = 500$ , max. 100 beam iterations or 60 sec

## NN design choices

- 2 or 3 hidden layers
- 5 or 10 neurons per hidden layer
- activation: tanh, ReLU, sigmoid
- features:  $F_1$ ,  $F_2$ ,  $F_3$

## Selected architectures (split by diff. $m$ )

| $m$ | layers  | act.    | features |
|-----|---------|---------|----------|
| 2   | 10-5-5  | ReLU    | $F_2$    |
| 3   | 10-10-5 | ReLU    | $F_2$    |
| 5   | 10-5-5  | sigmoid | $F_1$    |
| 10  | 10-5-5  | ReLU    | $F_2$    |

# Numerical results: Random and Real benchmarks

## Random benchmark

| comparison [Ensemble vs. IMSBS] | wins      | ties | losses |
|---------------------------------|-----------|------|--------|
| all groups (32)                 | <b>20</b> | 8    | 4      |
| $m = 2, 3$                      | <b>10</b> | 5    | 1      |
| $m = 5, 10$                     | <b>10</b> | 3    | 3      |

Wilcoxon:  $p = 0.009$  on all groups.

## Real benchmark

| comparison                     | wins      | ties | losses |
|--------------------------------|-----------|------|--------|
| LIMSBS-Ensemble vs. IMSBS (20) | <b>12</b> | 7    | 1      |

Comparable runtime; larger gains on harder instances such as Virus.

## Takeaway

The learned ensemble improves solution quality consistently, while keeping the original IMSBS machinery almost unchanged.

# Conclusion

- Learning an improved guidance promising (best-performing heuristic)
  - statistically significantly better results w.r.t. the literature best approach
- Low alphabet diversity, requires even stronger exploration/**richer feature portfolio**.

Thank you for your attention!

# Ensemble-IMSBS vs. exact DP-1 solution ( $m = 2$ )

$$|\Sigma| = 2$$

| $n$ | Avg. Gap     |
|-----|--------------|
| 50  | 1.0%         |
| 100 | 6.5%         |
| 200 | 10.0%        |
| 500 | <b>24.4%</b> |

Performance degrades as  
the instance size increases.

$$|\Sigma| = 4$$

| $n$ | Avg. Gap    |
|-----|-------------|
| 50  | 0.7%        |
| 100 | 0.3%        |
| 200 | 1.1%        |
| 500 | <b>2.0%</b> |

Near-optimal performance  
across all instance sizes.

**Ensemble-IMSBS stays within 2% of DP-1  
for all tested  $|\Sigma| = 4$  instances.**